

Bsc Final Year Project Retrospective
Full unit Project
A concurrency Based Game Environment

Henry Frodsham - 101050430
Royal Holloway University of London
Department of Computer Science
Supervisor: Felix Bradley

December 2025

Contents

1	Comparison to initial Project plan	3
1.1	Major Difference 1: Event architecture implementation schedule	3
1.2	Major Difference 2: ECS registry class implementation	3
1.3	Major Difference 3: Split screen implementation	4
1.4	Major Difference 4: the absence of game functionality	4
2	Reflection	5
2.1	What went well	5
2.2	What went wrong and why	5
2.3	Considerations for future development	6
3	Term 2 Timeline	7
4	Risk assessment and mitigation	8
4.1	Risk 1: imbalance between keyboard and controller	8
4.2	Risk 2: inadequate use of concurrency	8
4.3	Risk 3: homogenous testing environment	8
4.4	Risk 4: inability to implement REA	9
5	Acronomys	10
6	Glossary	11
7	References	12

1 Comparison to initial Project plan

The development of this project throughout term 1 has differed significantly compared to the original project plan.

1.1 Major Difference 1: Event architecture implementation schedule

The timeline for implementing event architecture outlined in my project plan is as follows:

Week 2: research efficient event architecture

Week 4: code central event bus

Week 6: implement event queue

whereas the actual order of implementation during the development of the project:

Week 1: research efficient event architecture

Week 2: implement event bus and event queue architecture

The large difference between the actual implementation order of event architecture is due to underestimating how crucial event architecture would be in my project. when initially creating the project plan I hadn't anticipated the role Event architecture would play in class communication within the project, for example with regards to input handling I expected each game instance to both read and process game input when in the currently implemented input handling I have a single class reading input that dispatches raw input to each game instance.

1.2 Major Difference 2: ECS registry class implementation

The anticipated timeline for implementing an ECS registry class is as follows:

Week 4: code ECS registry class

Week 7: code entity factories and create entity components for ECS

whereas the actual implementation order for creating the ECS registry class is as follows

Week 11: Create an ECS registry class with demo implementation

Week 11: create ECS components with an ECS factory

Although the timeline gap seems large it is not too different from the development progression I had in mind. I knew that ECS would be implemented closer to the end of the development for this project to produce a demonstration of the technology with a sample game object, so the major timeline gap is largely due

to underestimating the architectural groundwork required to implement input handling and a display overlay before reaching ECS implementations.

1.3 Major Difference 3: Split screen implementation

the anticipated timeline for implementing split screen within my project is:

Week 6: code initial split screen functionality

whereas the actual timeline throughout term 1 was:

Week 12: initial split screen functionality

A large factor contributing to this 6 week gap between anticipated timeline and actual timeline is the due to delays in implementing other functionality within my project, implementing split screen without a visible object on screen would be pointless so not only did ECS have to be implemented but so did a means of creating ECS entities easily and making those objects visible on the screen. another reason split screen functionality was delayed was due to a large amount of time spent when implementing an overlay system, I had expected this to be a menial task due to the relative ease of Dear Imgui although this turned out to not be the case so took a much larger stretch of time.

1.4 Major Difference 4: the absence of game functionality

the anticipated timeline for implementing a partially functional game is as follows:

Week 8-9: create partial game functionality

creating partial game functionality throughout term 1 was unfeasible due to delays associated with other components of this project, namely the overlay system and proper input handling. despite this, a delay to implementing game functionality may have been beneficial to my project due to recognising a new technology (Resource Entity Action) meaning creating game functionality next term 2 will be more cleanly integrated with the technology.

2 Reflection

2.1 What went well

development of this project throughout term 1 had multiple achievements that went past my initially set goals, these achievements involve either more robust implementations or extra functionality not considered during initial planning. I'm satisfied with the current overall state of the program as many large architectural tasks have been implemented in preparation for term 2 such that implementing RTS functionality will require less groundwork.

input handling was a success throughout the development in term1. In the currently implemented input handling, the user can unplug their controller whilst the project is running and then reconnect any time to resume control of their allocated viewport. In addition, the program can automatically recognise newly plugged in devices and configure them accordingly.

another success throughout term 1 was split screen functionality. The program can automatically create new viewports when new devices are connected and resize existing viewports as they are added or removed. the currently implemented split screen functionality also maintains a consistent aspect ratio for the displayed game world without scaling issues despite me expecting to encounter stretching of game objects depending on the size and shape of the viewport.

2.2 What went wrong and why

During development of this project in term 1 there were some interrupts that caused significant delays in following the development timeline. These interrupts did not halt development entirely and alternative solutions were found.

one major interrupt in term 1 was implementing an overlay system. the initial plan was to use Dear ImGui as an overlay system for this project however development proved that Dear ImGui is extremely difficult to integrate into Ogre3d. The main reason for this oversight is that at a surface level Ogre3d appears to support Dear ImGui natively and includes examples on their repository, the major issue encountered was that the given example assumes use of Ogre3d's ApplicationContextBase system (a means of simplifying Ogre3d applications). The underlying problem then was that implementing ApplicationContextBase would undermine this project's control over Ogre3d, not only triggering a refactor of preexisting code but would also affect my ability to fine tune Ogre3d's rendering to better fit the concurrency focus.

another major interrupt in term1 was the result of a limitation in SDL2, the controllers used to test the input handling in my project were generic off-brand controllers without a unique vendor id or product id. the absence of a vendor or product id in the controllers used led to an unforeseen issue where distinguish-

ing between each controller after they have been disconnected is impossible, this issue doesn't impact the behaviour of the program whilst the controllers are already connected however two controllers disconnecting at the exact same time can lead to a misallocation once both are reconnected.

2.3 Considerations for future development

after reflecting on the deficiencies of my previous plan I've realized more research and analysis of open source programs implementing technologies is required to prevent delays attributed to unforeseen issues. if i analysed other implementations of Dear imgui with ogre3d before starting development then the disruption could've been avoided entirely or dealt with in a more timely manor.

A major lesson learned from Term 1's development is that more careful testing is required across various hardware. SDL2's limitation where generic controllers cant be distinguished was only realised due to my use of generic controllers, if it was the case that i was using branded controllers this limitation would have gone unnoticed and would've had major implications for end users in the future.

another aspect ill consider during term 2s development is a more spaced out workload of writing final reports, throughout term 1 the writing of reports was inconsistent and only the project diary and control flow diagram where written as planned. to ensure this doesn't happen again I will write sections of the report alongside the diary, making sure to set sections of the report as branch objectives before merging into main.

3 Term 2 Timeline

following my reflection of how the initial project timeline translated into actual implementations throughout term 2, i have produced a timeline for term 2

Week 1: revisit term 1 implementation, refactor threading model and class communication

Weeks 1-2: Research REA implementations, implement a basic scenario with multiple entities that can perform basic actions.

Weeks 3-4: create a game world and a basic test scenario where each player is given starting units and starting territory

Weeks 4-5: model and create simple REA actions for taking territory, attacking units, placing units.

Weeks 6-7: create an undecorated UI, including context menus for actions that change depending on where the cursor is. settings and main menu screen

Weeks 7-8: perform market analysis on experienced real time strategy players and implement suggestions as prototypes

Weeks 8-9: perform further surveys of prototype combinations of these suggestions to produce the unrefined playable game

Weeks 9-12: produce a shader that complements the simplistic mesh's, produce a simplistic main menu screen and UI design to produce the finalised product.

(Time points overlap with each other to account for unexpected overhead)

4 Risk assessment and mitigation

4.1 Risk 1: imbalance between keyboard and controller

Real time strategy games typically aren't implemented with controller support for the reason of an unfair advantage between a player on keyboard and a player on controller. typically, the keyboard player can react faster and thus gains an edge in the RTS genre. to combat the imbalance between these 2 input devices I will design the attacking and expansion mechanics around ease of use for both input devices, the main differences i will implement differing to common RTS games is quick actions accessible through the controller D-pad which will update depending on the cursor position, e.g if the controller player is selecting inside their own territory then the quick actions could be "build x building here" or if they select an area close to a foe then "attack here".

4.2 Risk 2: inadequate use of concurrency

Throughout development in Term 2, it's possible that there's a major imbalance in processing between the game instance threads and the main application thread. whilst a slight imbalance is inevitable due to ogre3d running on the main thread there should still be a clear spread of processing load over the game instance threads. to ensure adequate spreading of workload i will monitor cpu core utilization whilst running prototype implementations of my project and adjust my implementation of concurrency to produce a relatively even spread of CPU workload across each core.

4.3 Risk 3: homogenous testing environment

This risk will become more important to avoid through this project's development in term 2. it's imperative that the performance of my project doesn't differ more than expected when running across different platforms or hardware combinations. to mitigate this risk, I will test this project across 2 different computers with one running Windows 11 and one running Linux, the hardware of each computer can be seen below:

Primary Testing and Development environment

Operating System: Windows 11 Home edition

CPU: Ryzen 9 9950X3D

GPU: Nvidia RTX 5090

RAM: 64GB DDR4

disk running and storing project: Nvme SSD

secondary testing environment

Operating System: Linux Mint with Wine installed

CPU: Ryzen 7 3700x

GPU: AMD Radeon 7700XT

RAM: 16GB DDR4

disk running and storing project: standard sata SSD

(both running at factory clock rates)

4.4 Risk 4: inability to implement REA

This risk is especially important for Term 2's development due to the previous failure to implement Dear ImGui. to prevent repeating the resulting delay to the development Timeline I will research REA implementations thoroughly and look for projects implementing both REA and ECS that are more relevant to my projects development and note which REA C++ library is standard for the entt library (for ECS). In the case of a previously unseen issue arising between ECS and REA compatibility I will research common workarounds or alternatives to implement instead, thus minimizing the final delay to the project timeline.

5 Acronomys

ECS - Entity Component System

REA - Resource Entity Action

RTS - Real Time Strategy

SDL2 - Software Direct Media Layer

LCM - Local Multi Player

CPU - Central Processing Unit

GPU - Graphical Processing Unit

RAM - Random Access Memory

6 Glossary

Ogre3d - a lightweight rendering engine in the form of a c++ library

Dear Imgui - a c++ rendering library focused on creating debug overlays

Software Direct Media Layer - a commonly used cross platform input handling library for c++

Resource Entity Action - a design pattern for real time strategy games that models entity interactions into transformation matrices that define resource exchanges between entities for fast evaluation

Entity Component System - a software architecture pattern that separates game objects into entities (identifiers) and components (pure data containers). ECS is used where composition is prioritized over inheritance

entt - an Entity Component System library for C++ that uses sparse set to store entities

7 References